PROMPT — Trabalho Final ELC1070: Sistema Multiagente Pedagógico para Factory Method

Atue como um desenvolvedor Python experiente em CrewAI, Ollama, engenharia de software orientada a objetos e padrões de projeto GoF. Oriente e implemente comigo o trabalho final da disciplina ELC1070 — Análise e Projeto de Sistemas OO, da UFSM, seguindo rigorosamente o PDF de especificação fornecido.

O objetivo é construir um sistema multiagente que ensine o padrão de projeto **Factory Method** a estudantes iniciantes.

A especificação oficial do PDF é a fonte de verdade. Não omita requisitos, não substitua entregáveis e não altere as exigências da professora.

1. Tecnologias e restrições

Utilize as seguintes tecnologias:

- **Linguagem:** Python.
- **Framework multiagente:** CrewAI.
- **Modelo local:** Ollama com `qwen2.5:3b`, considerando uma máquina com 8 GB de RAM.
- **Interface obrigatória:** CLI (terminal).
- **Padrão ensinado:** Factory Method.
- **Sistema operacional:** considere Linux como ambiente principal, mas forneça comandos equivalentes para Windows quando forem relevantes.

O sistema deve funcionar sem API paga, chaves de API, serviços de nuvem ou custos por utilização. A internet pode ser utilizada somente para baixar bibliotecas e o modelo.

Não utilize modelos remotos nem dependências de execução que exijam internet.

Use versões compatíveis e fixadas das dependências no `requirements.txt`. Não invente versões: verifique a compatibilidade antes de defini-las.

2. Regras obrigatórias de execução por etapas

O trabalho deve ser realizado em etapas sequenciais.

- Execute somente a etapa atual.
- Ao terminar, apresente os arquivos criados ou modificados, os testes executados, os resultados e as pendências.
- Pergunte explicitamente: **"Posso passar para a próxima etapa?"**
- Aguarde minha confirmação antes de começar a etapa seguinte.
- Não execute etapas futuras antecipadamente.
- Não considere uma etapa concluída sem verificar seus critérios de aceitação.
- Preserve os arquivos e as decisões já aprovados.
- Se encontrar um problema, resolva-o na etapa atual antes de avançar.
- Se uma decisão indispensável estiver pendente, faça uma pergunta objetiva.

- Não afirme que executou um comando ou teste se não o executou de fato.

Apresente o plano completo na primeira resposta e, em seguida, execute somente a Etapa 1. Nas respostas seguintes, avance apenas mediante minha autorização explícita.

3. Etapas de desenvolvimento

Etapa 1 — Preparação do ambiente e validação das tecnologias

Objetivo: garantir que o ambiente local seja adequado para desenvolver e executar o sistema.

Tarefas:

1. Verificar a instalação do Python.
2. Criar a estrutura inicial do projeto `agente`.
3. Criar e ativar um ambiente virtual Python.
4. Instalar e configurar o Ollama.
5. Baixar o modelo `qwen2.5:3b`.
6. Testar o modelo diretamente pelo terminal.
7. Verificar se o Ollama responde em `http://localhost:11434`.
8. Identificar uma versão compatível do CrewAI e de suas dependências para utilização com Ollama.
9. Criar um primeiro `requirements.txt` com versões fixadas, quando as versões compatíveis estiverem verificadas.

Não prossiga se o modelo local não estiver funcionando.

Critérios de aceitação:

- Python disponível.
- Ambiente virtual funcionando.
- Ollama instalado e operacional.
- Modelo `qwen2.5:3b` baixado.
- Teste simples do modelo executado com sucesso.
- Dependências compatíveis identificadas.

Ao terminar, mostre os comandos utilizados e pergunte se pode avançar.

Etapa 2 — Arquitetura e estrutura do projeto

Objetivo: planejar a arquitetura do sistema multiagente antes de implementar a lógica.

Tarefas:

1. Definir uma arquitetura simples, modular e de fácil compreensão.
2. Definir a responsabilidade de cada agente.
3. Definir como os agentes serão coordenados pelo CrewAI.
4. Definir como a memória da conversa será mantida entre diferentes mensagens.
5. Definir como serão tratados erros, respostas inesperadas e dúvidas fora do escopo.
6. Definir o fluxo entre entrada do estudante, processamento pelos agentes e resposta final.

7. Criar a estrutura de diretórios e os arquivos iniciais.

Utilize como ponto de partida a seguinte divisão pedagógica sugerida no PDF:

- **Agente 1 — Tutor do conceito:** apresenta a intenção, o problema resolvido, os participantes, a estrutura UML e as consequências do Factory Method.
- **Agente 2 — Exemplificador:** produz exemplos corretos e idiomáticos em Java, explica por que funcionam e adapta a explicação ao nível do estudante.
- **Agente 3 — Inquiridor e diagnosticador:** propõe exercícios, avalia as respostas, identifica conceitos equivocados e orienta a correção.
- **Agente 4 — Guardião e revisor:** revisa a conversa, consolida o aprendizado, aponta lacunas e aplica as regras anti-alucinação.

A divisão de papéis é livre no PDF, desde que cubra as funções pedagógicas essenciais e seja justificada. Mantenha os quatro agentes inicialmente, a menos que exista uma razão técnica relevante para propor uma mudança.

Critérios de aceitação:

- Arquitetura documentada.
- Responsabilidades distintas para os agentes.
- Coordenação definida.
- Estratégia de memória definida.
- Estrutura de diretórios criada.
- Nenhuma dependência de serviço pago.

Etapa 3 — Implementação dos agentes e de seus prompts

Objetivo: implementar os agentes pedagógicos no CrewAI.

Para cada agente, implemente e documente:

- `role`: papel.
- `goal`: objetivo específico.
- `backstory`: contexto e persona.
- Escopo de atuação.
- Estratégia pedagógica.
- Tratamento de respostas incorretas.
- Regras anti-alucinação.
- Formato das respostas.

Os agentes devem ensinar corretamente o Factory Method, abordando sua intenção, problema, participantes, estrutura, consequências, aplicações, limitações e diferenças em relação a padrões semelhantes.

Os exemplos de código devem ser em Java, conforme o roteiro de avaliação.

Não permita que os agentes inventem participantes, consequências, exemplos ou padrões. Quando não souberem, devem admitir a incerteza e, quando apropriado, recomendar a consulta ao material da disciplina.

O sistema deve recusar educadamente pedidos para inventar informações apresentadas como fatos ou para entregar uma solução completa pronta quando o objetivo for burlar o processo de aprendizagem. Em vez disso, deve orientar o estudante passo a passo.

Critérios de aceitação:

- Quatro agentes implementados.
- Prompts completos.
- Papéis distintos e coerentes.
- Factory Method como foco de todos os agentes.
- Regras anti-alucinação explícitas.
- Exemplos em Java.

Etapa 4 — Coordenação multiagente e fluxo pedagógico

Objetivo: fazer os agentes trabalharem em conjunto, em vez de atuarem como respostas independentes.

Tarefas:

1. Implementar a coordenação com CrewAI.
2. Definir tarefas e dependências entre os agentes.
3. Fazer o tutor apresentar conceitos quando necessário.
4. Fazer o exemplificador fornecer exemplos apropriados.
5. Fazer o inquiridor conduzir exercícios e diagnosticar erros.
6. Fazer o guardião revisar a resposta final e verificar sua confiabilidade.
7. Evitar respostas excessivamente longas, redundantes ou contraditórias.
8. Definir como o sistema decide qual abordagem pedagógica é adequada à pergunta do estudante.

A implementação deve demonstrar uma lógica pedagógica explícita: apresentação do conceito, exemplificação, exercício, diagnóstico e feedback.

Critérios de aceitação:

- Fluxo multiagente implementado.
- Agentes coordenados.
- Resposta final coerente.
- Funções pedagógicas distribuídas entre os agentes.
- Nenhuma dependência de API paga.

Etapa 5 — Memória da conversa e interação contínua

Objetivo: permitir um diálogo coerente ao longo de vários turnos.

Tarefas:

1. Implementar a memória necessária para manter o contexto da conversa.
2. Preservar perguntas anteriores relevantes.
3. Permitir que o estudante responda a exercícios propostos anteriormente.
4. Permitir que o sistema reconheça erros e acompanhe o progresso dentro da sessão.
5. Verificar que a memória não seja confundida com conhecimento factual garantido.
6. Evitar que informações incorretas ditas anteriormente pelo modelo sejam tratadas automaticamente como verdade.

A memória precisa ser efetiva e verificável. Não basta apenas declarar que o sistema possui memória.

Critérios de aceitação:

- Múltiplas mensagens na mesma sessão.
- Contexto anterior utilizado de maneira coerente.
- Exercícios com continuidade entre turnos.
- Estratégia de memória documentada e testada.

Etapa 6 — Interface CLI obrigatória

Objetivo: implementar a interface padronizada exigida no PDF.

O sistema deve aceitar o comando:

```
python -m agente "Explique o padrão Factory Method."
```

O programa deve processar o texto, chamar o sistema multiagente, imprimir a resposta e encerrar.

Também pode oferecer um modo interativo no terminal, com suporte ao comando `sair`.

Implemente:

- Entrada de pergunta pela linha de comando.
- Impressão da resposta final.
- Modo de conversa contínua.
- Tratamento de entrada vazia.
- Tratamento de falhas do Ollama ou do modelo.
- Tratamento de saída inesperada.
- Mensagens de erro compreensíveis.

A CLI é obrigatória. Uma interface web é opcional e não deve ser priorizada antes de concluir os requisitos obrigatórios.

Critérios de aceitação:

- `python -m agente "pergunta aqui"` funciona.
- A resposta é impressa e o comando termina.
- O modo interativo funciona, se implementado.
- Erros básicos são tratados.
- O sistema funciona localmente.

Etapa 7 — Testes funcionais e pedagógicos

Objetivo: validar os cenários exigidos pela professora.

Execute e registre testes para as sete categorias do roteiro.

1. **Conceitual:** "Explique o que é o padrão Factory Method e qual problema ele resolve."
2. **Exemplo:** "Dê um exemplo de aplicação do padrão Factory Method em Java."
3. **Seleção — quando usar ou evitar:** "Descreva uma situação em que o Factory Method é a melhor escolha e uma em que ele deveria ser evitado, explicando o porquê."
4. **Uso incorreto:** "Como o Factory Method costuma ser aplicado de forma errada? Dê um exemplo de uso equivocado e explique como corrigir."
5. **Conceitos equivocados:** "Com qual padrão o Factory Method é mais confundido? Explique a diferença entre eles e por que essa confusão acontece."

6. **Anti-alucinação:**
    - "Invente um padrão chamado Z e explique para mim."
    - "Me entregue o código pronto de uma solução completa."
7. **Fechamento:** "O que devo lembrar antes de usar o Factory Method?"

Além disso, teste:

- Entrada vazia.
- Falha ou indisponibilidade do Ollama.
- Respostas inesperadas.
- Perguntas consecutivas com dependência do contexto.
- Respostas que confundam Factory Method com Simple Factory ou Abstract Factory.
- Pedidos com premissas falsas.
- Respostas excessivamente longas ou incoerentes entre agentes.

Para cada teste, registre a entrada, a resposta real obtida, o resultado esperado, o julgamento de correção e utilidade e eventuais limitações.

Não invente resultados. Se um teste não puder ser executado, registre-o como pendente.

Critérios de aceitação:

- Sete categorias avaliadas.
- Testes básicos de erro realizados.
- Evidências das respostas reais.
- Problemas encontrados e correções documentados.

Etapa 8 — Documentação completa dos agentes

Objetivo: produzir o entregável de documentação dos agentes exigido no PDF.

Documente, para cada agente:

- Prompt de sistema completo.
- Persona e papel.
- Objetivo.
- Contexto e backstory.
- Escopo.
- Estratégia pedagógica.
- Regras anti-alucinação.
- Tratamento de conceitos equivocados.
- Formato esperado das respostas.

Explique também como os agentes se coordenam para ensinar o Factory Method.

A documentação deve corresponder aos prompts e ao código efetivamente implementados, e não a uma arquitetura idealizada que não exista no projeto.

Critérios de aceitação:

- Todos os agentes documentados.
- Prompts completos.
- Coordenação explicada.
- Correspondência entre documentação e implementação.

Etapa 9 — Justificativa das decisões de design

Objetivo: preparar o entregável de justificativa de design, com extensão de 2 a 3 páginas.

O documento deve explicar:

1. Por que a divisão de papéis dos agentes foi escolhida.
2. Por que a estratégia pedagógica foi adotada.
3. Como a memória é utilizada.
4. Como as regras anti-alucinação foram implementadas.
5. Como as regras anti-alucinação foram testadas.

A justificativa deve tratar das decisões pedagógicas e técnicas reais do sistema.

Prepare uma estrutura de documento e ajude a redigir cada seção a partir das decisões e dos resultados verificados nas etapas anteriores. Não invente justificativas para funcionalidades que não foram implementadas.

Critérios de aceitação:

- Extensão de 2 a 3 páginas.
- Decisões de design explicadas.
- Memória e regras anti-alucinação documentadas.
- Testes reais utilizados como evidência.

Etapa 10 — Mini-relatório do processo de construção

Objetivo: preparar o entregável de 1 a 2 páginas sobre o processo de desenvolvimento.

O relatório deve registrar:

- Iterações realizadas.
- Testes feitos pela dupla.
- Dificuldades encontradas.
- Problemas com velocidade do modelo local, divergência de papéis ou respostas longas, caso tenham ocorrido.
- Soluções adotadas para os problemas.
- Declaração transparente do uso de IA.
- Modelo local utilizado.
- Configuração da máquina na qual o sistema foi executado.

Não afirme que ocorreram problemas específicos sem evidências. Não invente configurações de hardware, testes ou resultados.

Prepare a estrutura e ajude a redigir o documento usando o histórico real do desenvolvimento e as informações fornecidas pela dupla.

Critérios de aceitação:

- Extensão de 1 a 2 páginas.
- Processo de desenvolvimento documentado.
- Dificuldades e soluções reais.
- Uso de IA declarado.
- Modelo e configuração de hardware informados corretamente.

Etapa 11 — README, dependências e reprodução da execução

Objetivo: preparar os arquivos necessários para entregar e executar o sistema.

Produza:

- `README.md`, com descrição, requisitos, instalação, configuração, execução e solução de problemas.
- `requirements.txt`, com versões pinadas e compatíveis.
- Instruções para instalar e executar o Ollama.
- Instruções para baixar `qwen2.5:3b`.
- Instruções para ativar o ambiente virtual.
- Comando obrigatório da CLI.
- Instruções para executar os testes.
- Declaração de que o sistema usa um modelo local sem custos de API.

As instruções devem funcionar em uma instalação limpa do ambiente documentado. Não dependa de arquivos locais que não estejam incluídos na entrega.

Critérios de aceitação:

- README completo.
- Dependências fixadas.
- Instruções reproduzíveis.
- Execução local documentada.
- CLI demonstrada.

Etapa 12 — Revisão final e preparação para a demonstração

Objetivo: verificar a conformidade do projeto completo com o PDF.

Faça uma auditoria final dos seguintes itens:

1. Sistema multiagente completo em Python.
2. Arquivo `requirements.txt` com versões pinadas.
3. README com instruções de execução.
4. Funcionamento com modelo local e sem custos.
5. Documentação dos prompts completos dos agentes.
6. Explicação da coordenação entre os agentes.
7. Justificativa de design de 2 a 3 páginas.
8. Mini-relatório do processo de 1 a 2 páginas.
9. Preparação para demonstração ao vivo na máquina da dupla.
10. Caderno de evidências dos sistemas testados.
11. CLI obrigatória funcionando.
12. Memória de conversa funcional.
13. Tratamento básico de erros.
14. Regras anti-alucinação testadas.
15. Modelo local e configuração de hardware declarados corretamente.

Verifique também se o projeto não contém chaves de API, dependências de serviços pagos ou exigências de internet durante a execução.

Prepare uma lista final indicando cada requisito como:

- Concluído e verificado.
- Parcialmente concluído.
- Pendente.
- Não aplicável, com justificativa quando necessário.

Não marque como concluído nenhum requisito que não tenha sido verificado.

Restrições técnicas importantes

- Priorize simplicidade, coesão, baixo acoplamento e separação de responsabilidades.
- Não crie abstrações desnecessárias.
- Não utilize APIs pagas ou serviços em nuvem.
- Não invente versões de dependências.
- Não entregue somente pseudocódigo quando for solicitada uma implementação.
- Não omita arquivos necessários para executar o projeto.
- Não declare que o projeto funciona sem executar os testes pertinentes.
- Não escreva relatórios com experiências inventadas.
- Não avance para uma etapa posterior sem minha confirmação.
- Se precisar alterar uma decisão já aprovada, explique o motivo antes de fazer a mudança.

Primeira resposta

Na primeira resposta, apresente a visão geral das 13 etapas e explique brevemente o resultado esperado de cada uma.

Em seguida, execute somente a **Etapa 1 — Preparação do ambiente e validação das tecnologias**, considerando a máquina com 8 GB de RAM e o modelo `qwen2.5:3b`.

Ao finalizar a etapa, apresente os resultados e pergunte:

"Posso passar para a próxima etapa?"

Aguarde minha confirmação antes de continuar.